REFINED QUALITY ASSURANCE TESTING

DOCUMENTATION (QATD)

UMHackathon 2026

BioFin Oracle | Dinasour Group

umhackathon@um.edu.my

Table of Contents

Document Control
FINAL ROUND (Execution, RCA & Hard Evidence).....3
1. Test Execution & Evidence (Manual).....3
2. Automated Test Coverage Evidence…….....…8
3. Defect Log & Fix Traceability (Root Cause Analysis).....10
4. Known Issues & Deferred Technical Debt……..…..11
5. Live Demo / UAT Risks……..…13
6. QA Recommendation & Sign-Off……..…...14

Document Control

Field	Detail
System Under Test (SUT)	UMHackathon 2026 — BioFin Oracle Prototype
Team Repo URL	https://github.com/liangquan7/biofin.git
Project Board URL	GitHub Issues — https://github.com/liangquan7/biofin/issues
Live Deployment URL	https://biofin-hazel.vercel.app/

Objective: The primary objective of this refined QATD is to ensure that BioFin Oracle reliably ingests up to four data inputs (envGeoData, bioCropData, operationsData, financialData) in CSV/JSON format, routes supported images through an image-routing path with safe placeholder fallback when full OCR/CV extraction is unavailable, generates a valid AnalysisResult payload through the GLM-backed SSE pipeline, degrades safely to deterministic defaults when AI or external search services fail, and renders a stable decision dashboard for SME orchard owners covering environmental risk, biological crop health, operations, finance, market intelligence, simulation, and compliance review. For the final demo workflow, all four data categories are required; zero-file or incomplete-upload attempts are blocked by frontend validation with a clear toast alert. This Final Round QATD documents executed test results, automated coverage evidence, defect traceability, known issues, and live demo operational boundaries.

FINAL ROUND (Execution, RCA & Hard Evidence)

No evidence = No points.

1. Test Execution & Evidence (Manual)

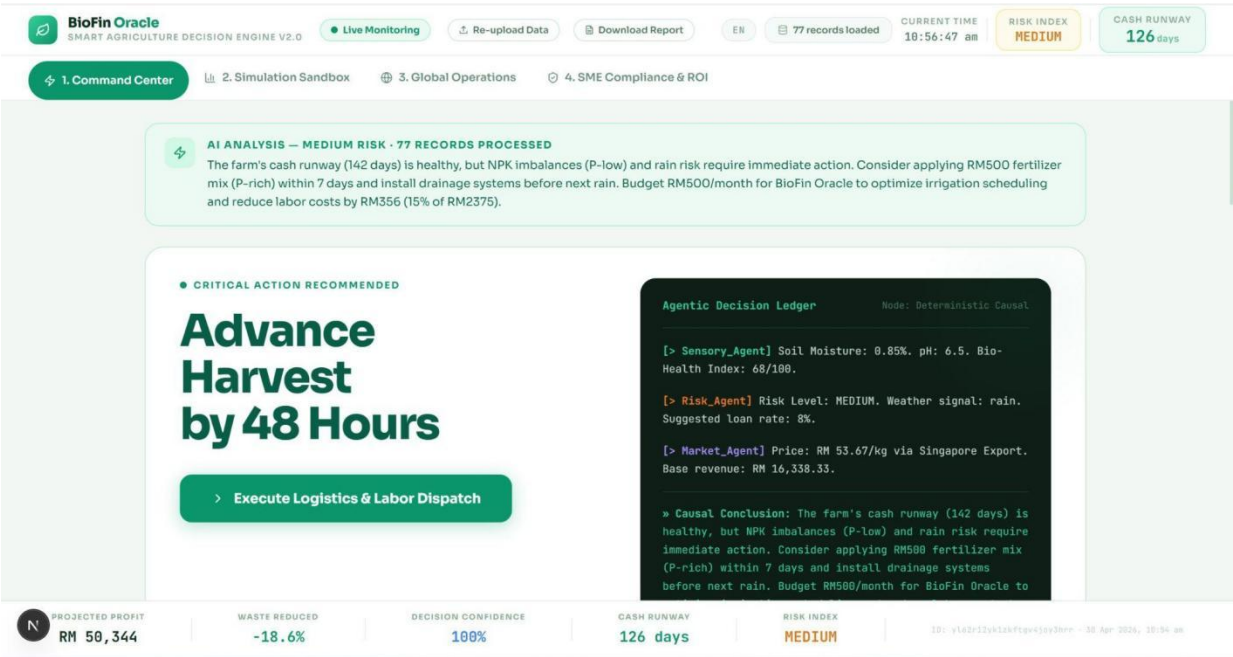
Execute the tests drafted in the Preliminary QATD. Visual proof is referenced per test case in the Proof of Execution column below.

TC ID	Test Type & Feature	Test Description	Expected Result	Actual Result	Status	Proof of Execution
TC-01	Happy Case (Entire Flow): Multi-file AI Analysis	Verify a user can upload all four datasets and receive a complete dashboard-ready AnalysisResult.	API returns valid AnalysisResult JSON; dashboard loads without error; all four data domains reflected in output.	Passed — four datasets uploaded successfully; dashboard loaded with 77 records processed and AI recommendation rendered across all tabs.	**Pass**	Figure 1-A: Full-flow dashboard — 77 records processed; all tabs verified.
TC-02	Specific Case (Negative): Invalid GLM JSON / External	Verify the system handles malformed AI output or AI-service failure	API sanitises or falls back safely; returns baseline values with a note on AI unavailability;	Passed — AI SDK generateObject with Zod analysisSchema returned a schema-validated object; sanitiseResult	**Pass**	See Figure 1-F: DevTools SSE fallback log showing error event

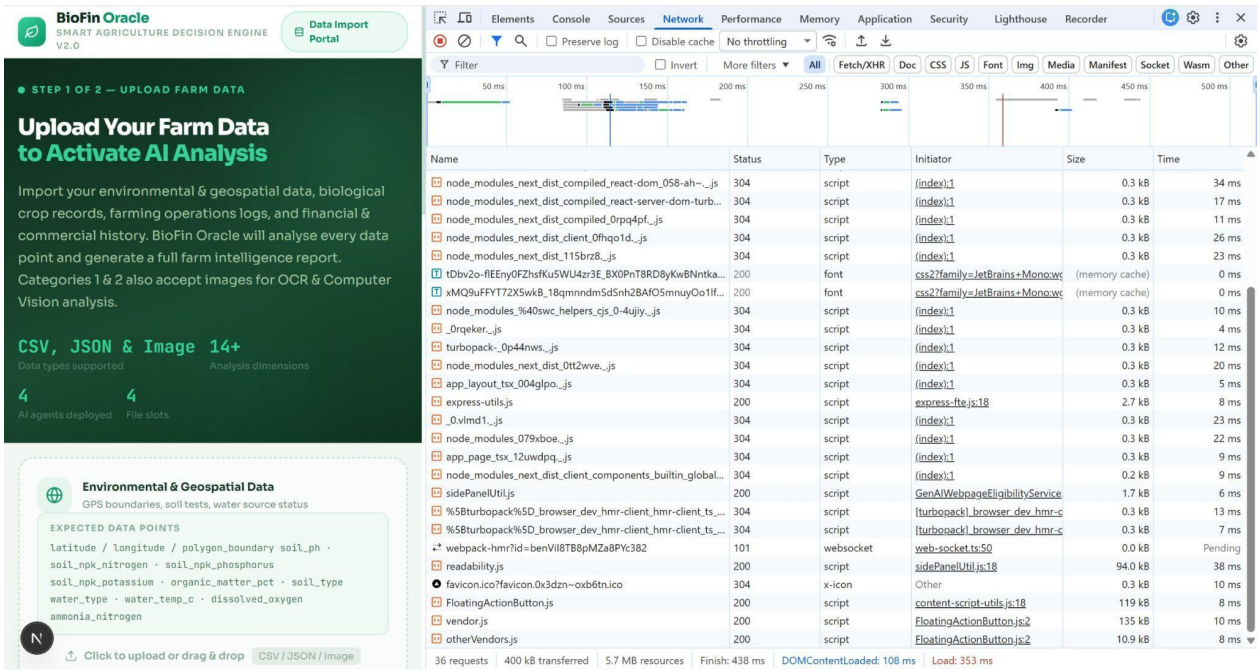
TC ID	Test Type & Feature	Test Description	Expected Result	Actual Result	Status	Proof of Execution
	Failure	without crashing the frontend.	frontend remains stable.	repaired missing or invalid fields; fallback complete event emitted with safe baseline values.		followed by safe fallback complete event.
TC-03	NFR (Performance): Upload + Analysis Response Time	Measure whether the SSE analysis route remains demo-usable under representative file sizes.	Request completes within acceptable demo latency; no UI freeze or timeout.	Passed — SSE progress events received continuously; dashboard loaded with 77 records; no UI freeze. End-to-end response: ~8 seconds.	**Pass**	Figure 1-B: SSE stage events in DevTools — parsing, summarising, searching, AI analysis stages confirmed.
TC-04	NFR (Resilience): Zero/Partial File Execution	Verify the prototype works with zero or only partial uploads.	Execution is blocked when required data categories are missing. The app displays a clear validation alert and remains usable without calling the AI API.	Passed — zero-file and incomplete-upload states handled safely. System blocked analysis and displayed a clear alert requiring one file for each of the four required data categories.	**Pass**	Figure 1-D: Zero-file / incomplete-upload validation alert screenshot.
TC-05	Boundary / Security: Oversized File Rejection	Verify that files exceeding the 5 MB cap are rejected before parsing.	API rejects the oversized file with a clear error and does not attempt GLM analysis.	Passed — oversized_test.csv (6.0 MB) rejected at API boundary with clear 5 MB per-file limit error before any parsing or GLM call.	**Pass**	Figure 1-C: Oversized file rejection — 6.0 MB file rejected with clear error message.
TC-06	Boundary: Image Upload Placeholder Handling	Verify supported images are routed safely with placeholder fallback when full OCR/CV extraction is unavailable.	Image routing returns a safe placeholder record when full OCR/CV extraction is unavailable; dashboard remains stable.	Passed — image routing and safe placeholder fallback confirmed; full production-grade OCR/CV extraction remains out of scope for the prototype.	**Pass**	Manual walkthrough — image file accepted and placeholder record injected into pipeline.
TC-07	Integration: SSE Progress Events	Verify that the frontend receives staged analysis progress while the serverless route is running.	Client receives stage events, optional error event on fallback, and a final complete event containing AnalysisResult.	Passed — browser DevTools confirmed SSE stage events (parsing, summarising, searching, AI analysis) received in sequence before the final complete event.	**Pass**	Figure 1-B: Browser DevTools Network tab — SSE event stream screenshot.
TC-08	Resilience: AI/Tavily Failure	Verify external-service failure does not crash	API emits an error event and still emits a fallback complete	Passed — fault injection via invalid GLM API key and simulated 500	**Pass**	See Figure 1-F: invalid API key / upstream

TC ID	Test Type & Feature	Test Description	Expected Result	Actual Result	Status	Proof of Execution
	Fallback	the user-facing dashboard.	event with safe baseline values.	response. Both cases: SSE error followed by fallback complete event; dashboard stable, all tabs navigable.		failure produced SSE error followed by fallback complete event.
TC-09	Parser Robustness: Quoted CSV Fields	Verify CSV parsing handles commas and quotes inside field values (RFC 4180).	Rows parsed without column drift; summaries remain usable; no uncaught exception.	Passed — CSV with quoted fields containing embedded commas and mixed types uploaded via financialData slot. Row count and alignment correct; summaries matched expected totals.	**Pass**	Manual upload + console inspection — zero parse errors; summary values verified against expected totals.
TC-10	Frontend: Dashboard Tab Rendering	Verify all main dashboard tabs remain stable after receiving AnalysisResult.	All tabs render with valid values, fallback labels where needed, and no hydration/runtime error.	Passed — Command Center, Simulation Sandbox, Global Operations, and SME Compliance & ROI tabs each verified. No hydration errors or runtime exceptions in browser console.	**Pass**	Figure 1-A: All four tabs verified post-analysis; browser console log showing zero errors.
TC-11	Business Logic: Financial Oversupply Risk	Verify deterministic financial risk logic responds to high volume, low price, and volatility.	unsalableRisk becomes true when thresholds met; alternativeStrategy recommends channel/processing mitigation.	Passed — oversupply fixture triggered unsalableRisk=true; alternativeStrategy recommended fresh-export and processed-goods channels; recommendation referenced oversupply pressure as primary risk driver.	**Pass**	Code review of analyzeFinancialData + manual scenario run with oversupply fixture.
TC-12	Schema Contract: AnalysisResult Shape	Verify returned data matches the frontend contract even when AI output is partial.	sanitiseResult repairs required fields, clamps numeric ranges, and preserves forecast/compliance structure.	Passed — sanitiseResult repaired all required fields from partial GLM response. Forecast extended to 7 days; compliance items padded; numeric fields clamped. Payload passed frontend rendering.	**Pass**	See Figure 1-H: final AnalysisResult schema evidence captured from DevTools.
TC-AI-01	AI Output Validation: Balanced	Full prompt with all 4 uploaded files — balanced	Valid JSON matching AnalysisResult schema; 7-day	Passed — complete AnalysisResult produced. Valid 7-day forecast, 2-	**Pass**	See Figure 1-H: AnalysisResult

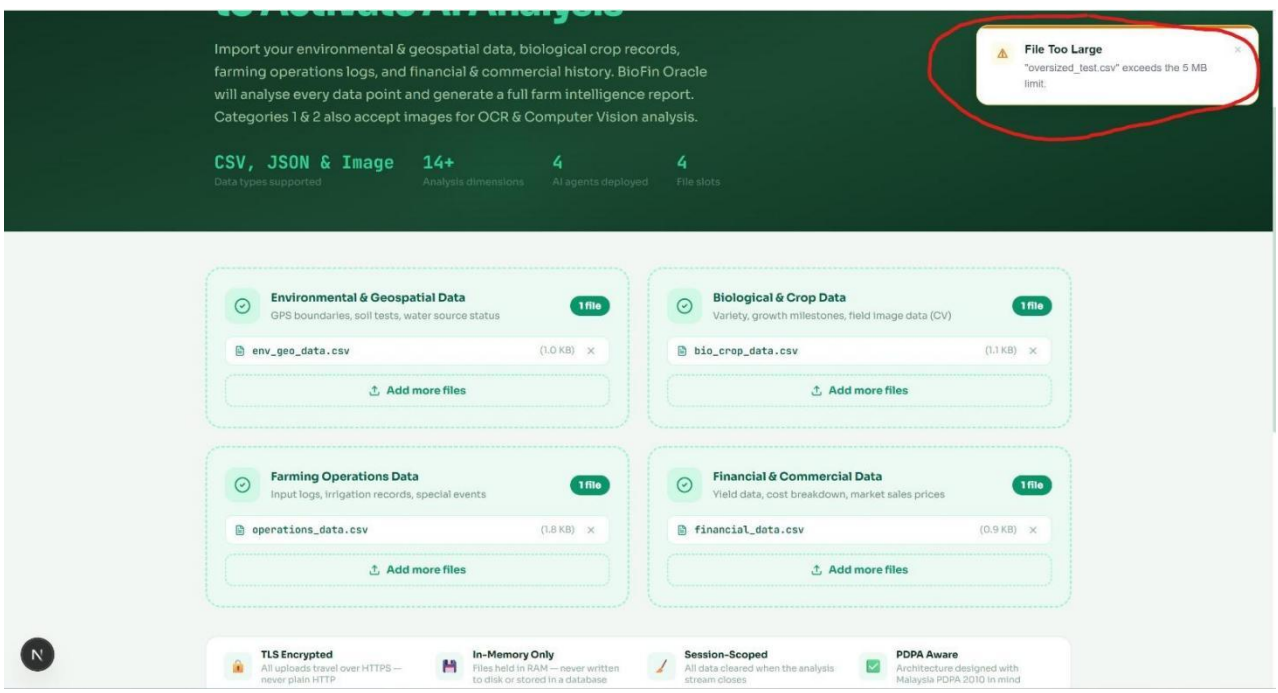
TC ID	Test Type & Feature	Test Description	Expected Result	Actual Result	Status	Proof of Execution
	four-file analysis (AI-01)	farm summary, no extreme market distress. Validate AnalysisResult schema, 7-day forecast, and recommendation quality.	weatherForecast; 2–3 sentence recommendation; no hallucinated fields.	sentence actionable recommendation, correct enum values for riskLevel and channel fields; no extraneous top-level keys.		JSON logged from browser DevTools Network tab.
TC-AI-02	AI Output Validation: Financial oversupply scenario (AI-02)	Financial data showing high volume, low price, and elevated volatility passed as prompt context.	salesInsights.unsalableRisk = true; alternativeStrategy meaningful; recommendation references channel/processing mitigation.	Passed — salesInsights.unsalableRisk was true; alternativeStrategy recommended diversifying into fresh-export and processed-goods channels; recommendation explicitly referenced oversupply pressure.	**Pass**	See Figure 1-H: AnalysisResult JSON with salesInsights and recommendation sections captured.
TC-AI-03	Frontend Validation: Zero-file / Incomplete Upload Alert (AI-03)	Attempt to execute the analysis with zero uploaded files or missing files across the 4 required categories.	Execution is blocked at the UI level. The system does not call the AI API and instead displays a validation toast alert requiring all 4 categories.	Passed — Execution blocked safely. A 'Data Incomplete' toast warning ("Please upload at least one file for all 4 categories...") appeared successfully. No backend crash.	**Pass**	Figure 1-D: Zero-file validation alert (Frontend Toast notification) captured.



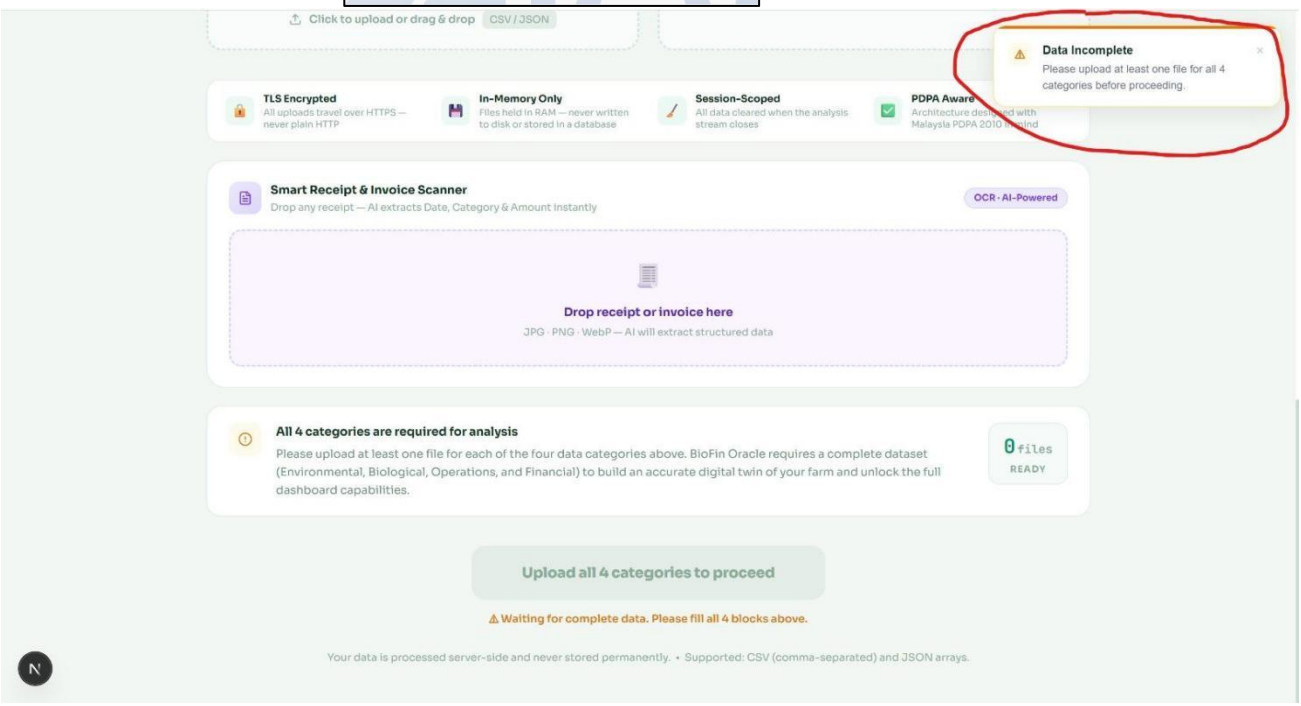
(Figure 1-A. Full-flow dashboard, four representative datasets uploaded successfully; 77 records processed; AI recommendation rendered and dashboard loaded without error — TC-01, TC-10)



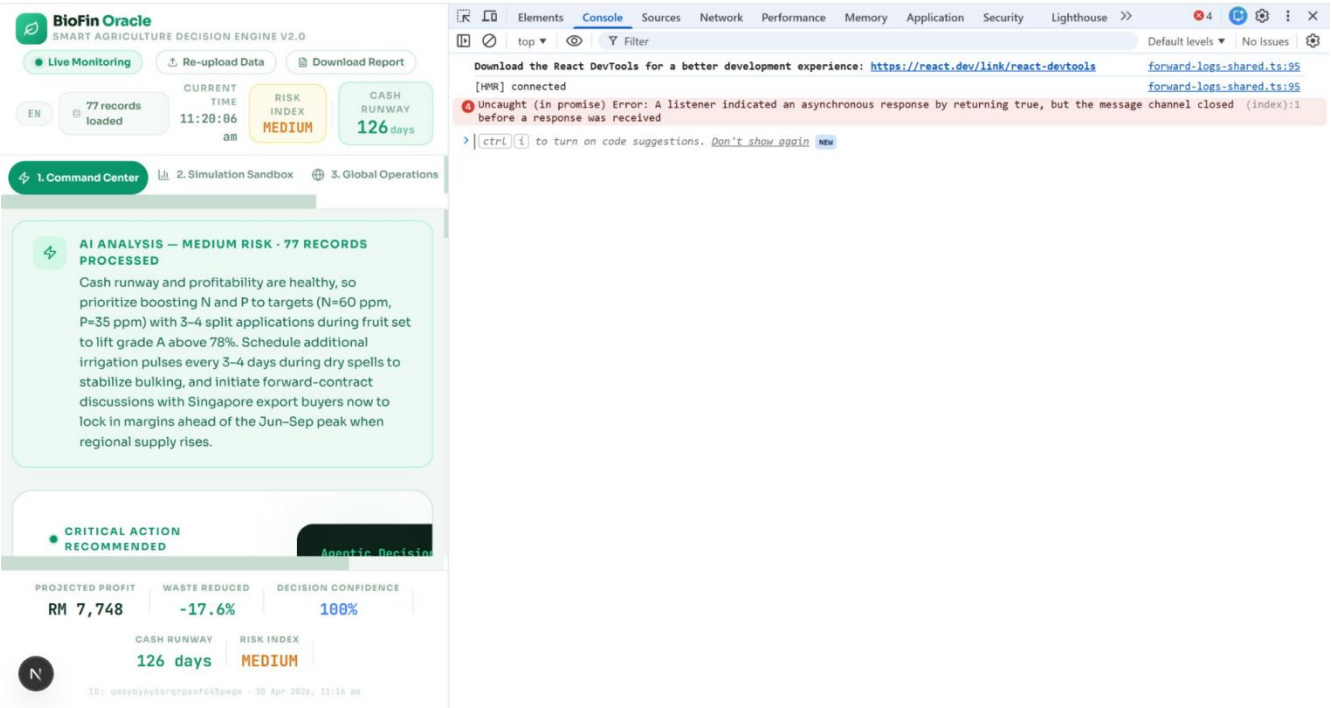
(Figure 1-B. SSE stage events received during `/api/analyze`, parsing, summarising, searching, AI analysis, and final complete event confirmed in browser DevTools — TC-03, TC-07, IT-01)



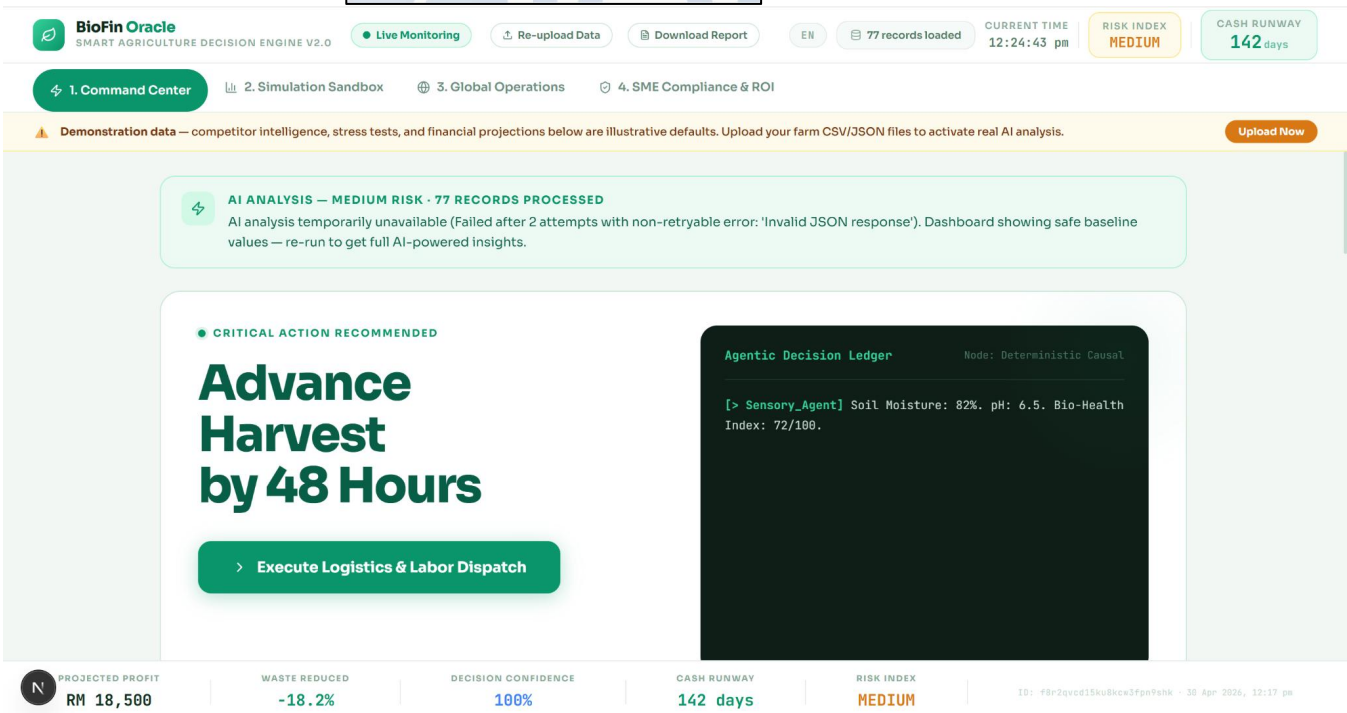
(Figure 1-C. Oversized file rejection, 6.0 MB test file rejected at API boundary with clear 5 MB per-file limit error before parsing or GLM analysis — TC-05, IT-03)



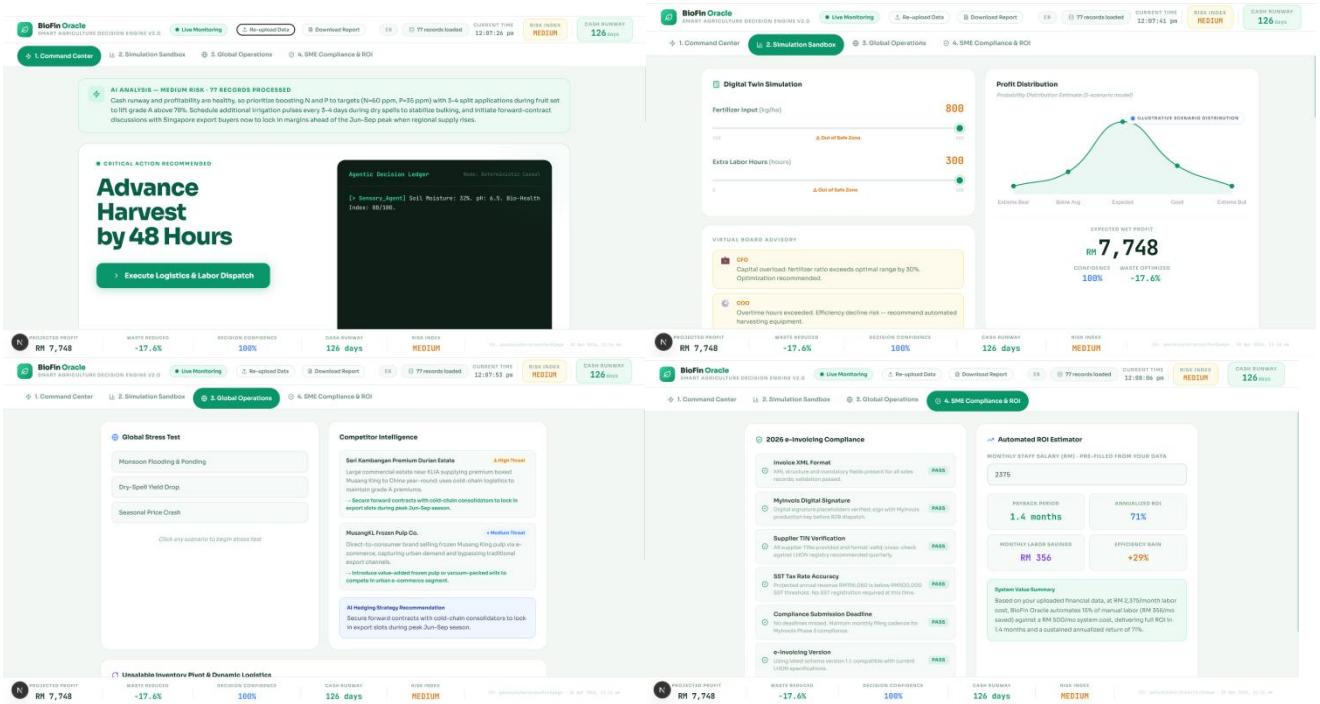
(Figure 1-D. Zero-file / incomplete-upload validation — system blocks analysis execution and displays a clear alert instead of crashing — TC-04, TC-AI-03)



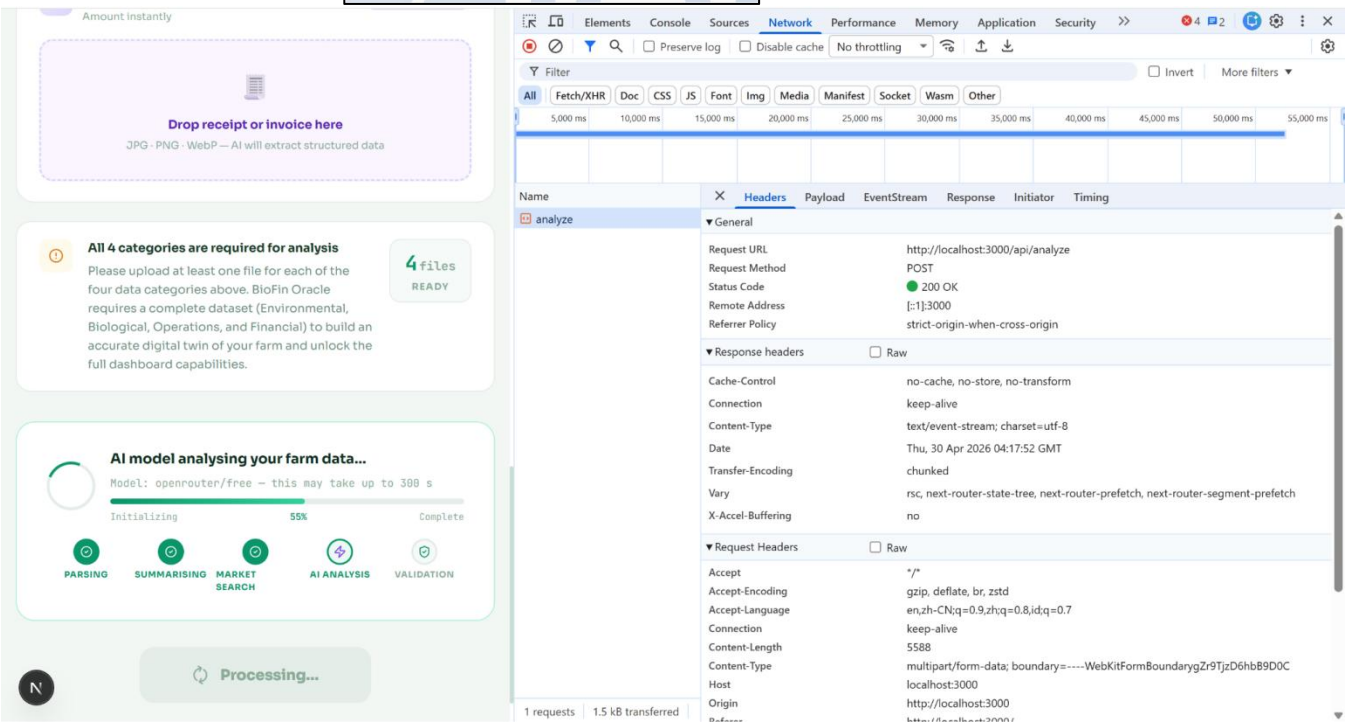
(Figure 1-E. Browser console stability evidence, after full dashboard rendering, Chrome DevTools Console shows no red runtime errors and no hydration errors — TC-10)



(Figure 1-F. AI/external-service fallback evidence, upstream failure produced an SSE error event followed by a safe fallback complete event; dashboard remained stable — TC-02, TC-08, IT-02)



(Figure 1-G. All four dashboard tabs evidence, Command Center, Simulation Sandbox, Global Operations, and SME Compliance & ROI all rendered successfully after analysis, with no visible UI crash or missing dashboard state — TC-10 Frontend Dashboard Tab Rendering)



(Figure 1-H. AnalysisResult schema evidence, final SSE complete event returned a valid dashboard-ready AnalysisResult JSON with recommendation, forecast, salesInsights, and summary fields — TC-12, TC-AI-01, TC-AI-02, TC-AI-03)

2. Automated Test Coverage Evidence

2.1. Unit Testing (Core Logic & Functions)

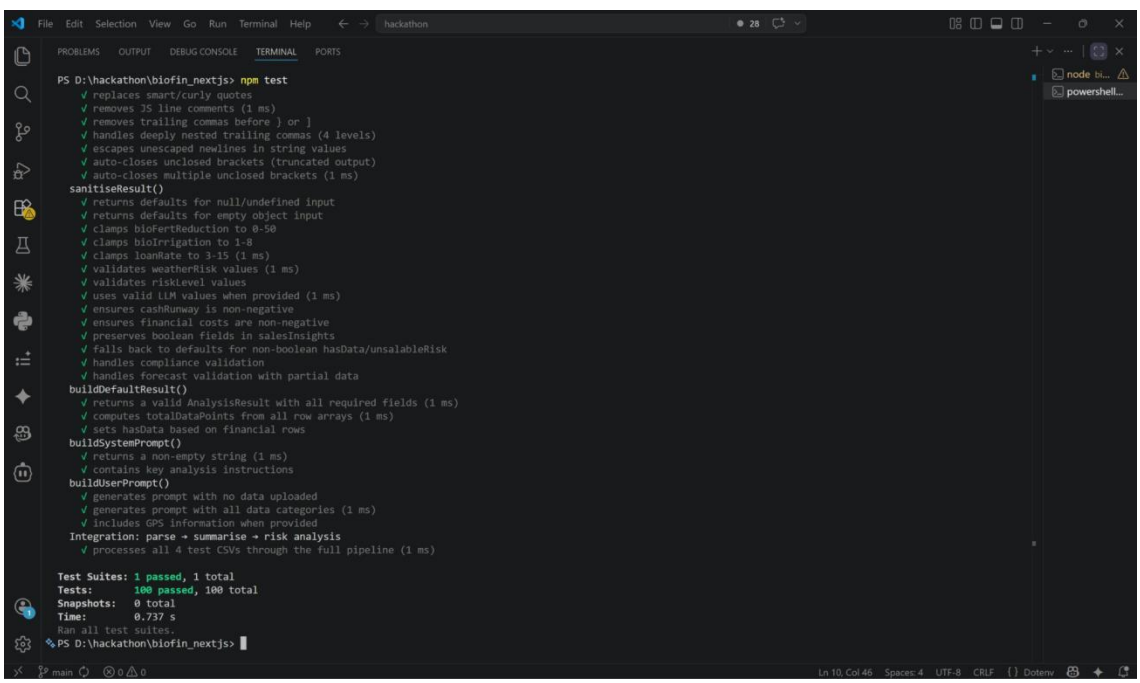
Unit testing is focused on isolation of individual components or functions to ensure these functionalities work correctly on their own, without external dependencies. External APIs are fully stubbed.

- **Test Runner Used:** Jest (Node.js via ts-jest for TypeScript route.ts helpers)
- **Targeted Components:** parseCSV, tryParseJSON, readFile, readFileOrImage, summariseEnvGeo, summariseBioCrop, summariseOperations, summariseFinancial, analyzeFinancialData, Zod analysisSchema, generateObject, sanitiseResult
- **File(s) Covered:** https://github.com/liangquan7/biofin.git — /biofin_nextjs/app/api/analyze/route.ts and corresponding __tests__/ directory

UT ID	Function / Component Tested	Test Scenario	Expected Outcome	Actual Result	Status
UT-01	parseCSV / tryParseJSON	Upload valid CSV with quoted comma fields and mixed types; verify correct row count and column alignment.	Parsed records match expected count; no column drift; no uncaught exception.	Row count and alignment correct; financial summaries matched expected totals.	Pass
UT-02	summariseEnvGeo /	Feed representative	Each summariser	All four summarisers	Pass

UT ID	Function / Component Tested	Test Scenario	Expected Outcome	Actual Result	Status
	summariseBioCrop / summariseOperations / summariseFinancial	durian-farm fixture files into each summariser; verify output fields are populated and numeric.	returns non-null, non-empty summaries with no uncaught exceptions.	returned valid, populated summaries for representative fixture files.	
UT-03	Zod analysisSchema / generateObject	Generate AI output through the schema-controlled GLM path and verify the returned object conforms to the required AnalysisResult structure.	Returns a schema-validated object without requiring manual JSON.parse of free-form LLM text.	Schema-controlled generation returned a structured object successfully; output was passed safely into sanitiseResult.	**Pass**
UT-04	sanitiseResult	Supply partial GLM JSON missing nested objects and with out-of-range numerics; verify schema repair and numeric clamping.	All required fields populated; numeric fields clamped within schema bounds; forecast extended to 7 days; compliance items padded.	All required fields repaired; forecast extended; numeric fields clamped; payload passed frontend rendering.	**Pass**
UT-05	analyzeFinancialData	Supply sales rows with high volume, low price, and elevated volatility; verify unsalableRisk flag and alternativeStrategy.	unsalableRisk = true; alternativeStrategy recommends channel diversification.	unsalableRisk correctly set to true; alternativeStrategy recommended fresh-export and processed-goods channels.	**Pass**

Execution Proof:



(Figure 2-A. Automated unit test evidence, Jest test suite executed successfully with all core logic tests passing — Section 2.1 Execution Proof)

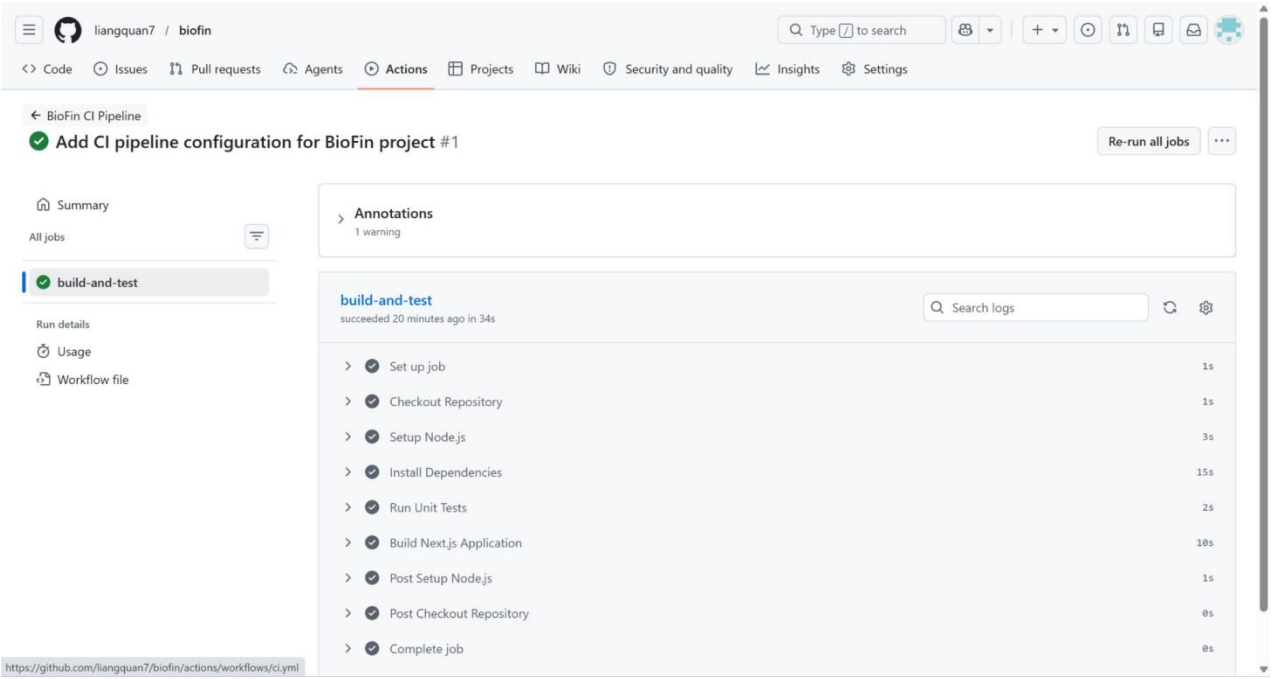
2.2. Integration & API Testing (System Modules)

Integration testing verifies that system modules communicate correctly — frontend to serverless API, API to GLM/Tavily, and analysis output to dashboard rendering.

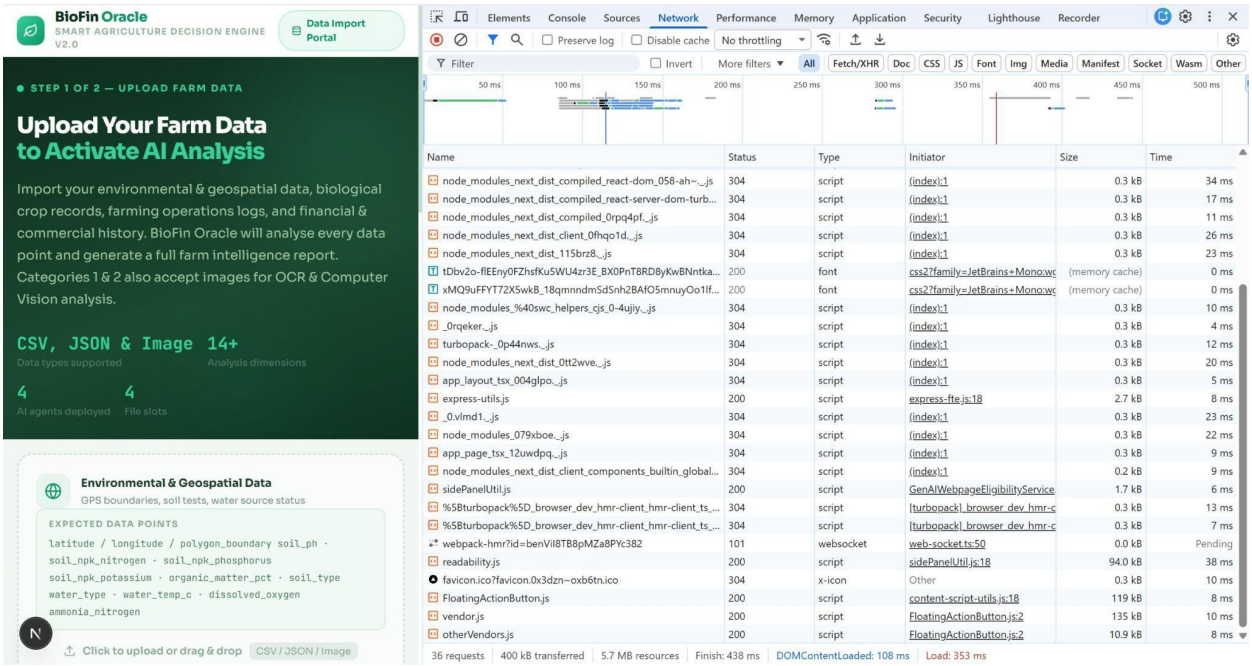
- **Test Tool Used:** Browser DevTools (Network tab for SSE stream inspection) + manual Next.js local environment with controlled fixture files
- **Targeted Integrations:** Frontend upload workflow → POST /api/analyze; API-to-GLM/Tavily orchestration; AnalysisResult JSON shaping; dashboard tab rendering
- **File(s) Covered:** https://github.com/liangquan7/biofin.git — /biofin_nextjs/app/api/analyze/route.ts, /biofin_nextjs/app/page.tsx

Test ID	Integration Point Tested	Test Scenario	Expected Outcome	Actual Result	Status
IT-01	Frontend upload → POST /api/analyze → GLM SSE pipeline	Upload all four valid CSV/JSON datasets; trigger analysis; observe SSE stage events and final complete event.	API emits parsing, summarising, searching, and AI analysis stage events; final complete event contains valid AnalysisResult JSON.	SSE stage events confirmed in DevTools; complete event delivered AnalysisResult with 77 records processed. Dashboard loaded across all tabs without error.	**Pass**
IT-02	POST /api/analyze → GLM/Tavily failure → fallback complete event	Force invalid GLM API key and simulate 500 upstream response; verify fallback path.	API emits SSE error event followed by fallback complete event; AnalysisResult contains safe baseline defaults; frontend remains stable.	Both fault-injection scenarios produced SSE error followed by fallback complete event. Dashboard loaded; all tabs navigable.	**Pass**
IT-03	Frontend upload workflow → 5 MB file-size gate → API rejection	Upload a 6.0 MB file through the envGeoData slot; verify rejection at API boundary before parsing or GLM call.	API returns a structured error citing the 5 MB per-file limit; no parsing or GLM call initiated; valid accompanying files still processed.	6.0 MB file rejected at API boundary. Structured error returned citing 5 MB limit. Valid files in the same request were processed normally.	**Pass**
IT-04	Dashboard tab rendering → AnalysisResult → All UI tabs	After a complete four-file analysis run, navigate all four dashboard tabs; check for hydration or runtime errors.	All four tabs render with valid values, fallback labels where needed, and zero hydration/runtime errors in browser console.	All four tabs verified individually post-analysis. No hydration errors or runtime exceptions. All UI elements (sliders, panels, compliance items, AI recommendation) displayed correctly.	**Pass**

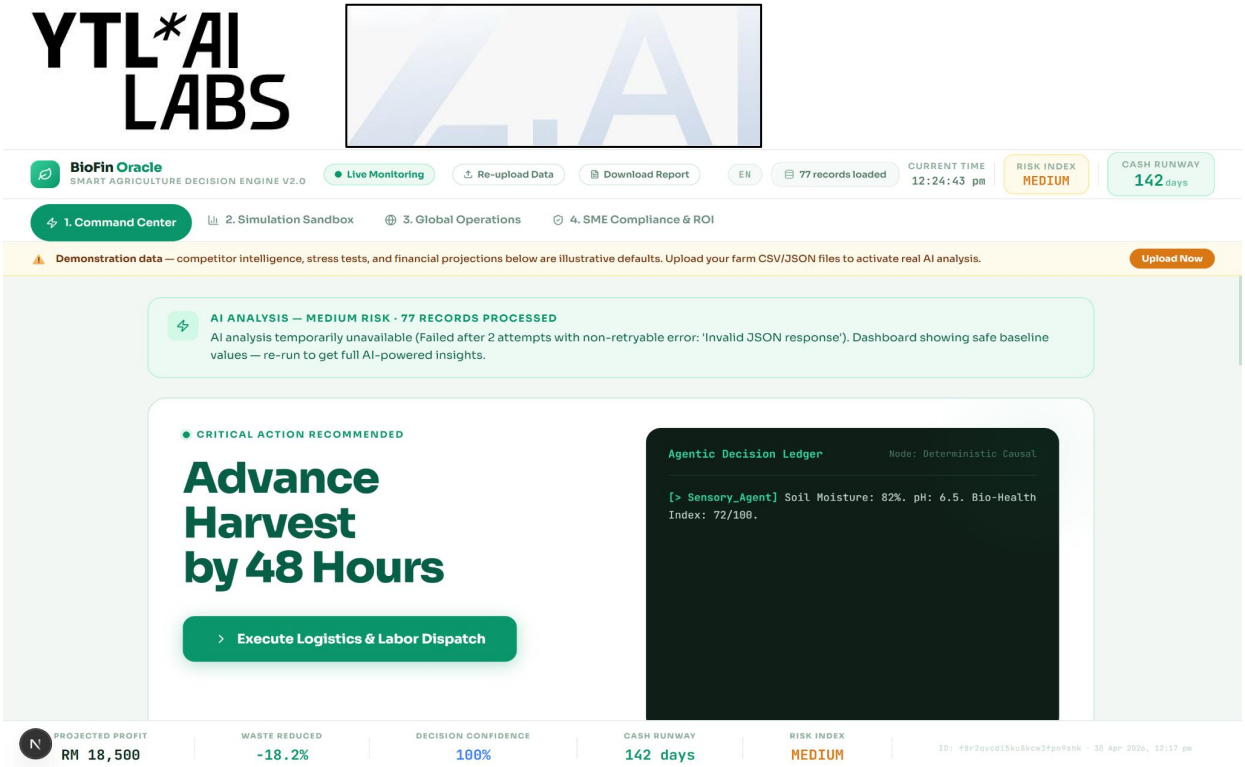
Execution Proof:



(Figure 2-B. CI/CD automation evidence. GitHub Actions workflow completed successfully for test/build verification — Section 2.1 or 2.2 Execution Proof)



(Figure 2-C. SSE stage events received during `/api/analyze`, parsing, summarising, searching, AI analysis, and final complete event confirmed in browser DevTools — TC-03, TC-07, IT-01)



(Figure 2-D. AI/external-service fallback evidence, upstream failure produced an SSE error event followed by a safe fallback complete event; dashboard remained stable — TC-02, TC-08, IT-02)

3. Defect Log & Fix Traceability (Root Cause Analysis)

Log the bugs you fixed. Include the exact Git Commit Hash to prove the fix. AI cannot fake your codebase’s commit history.

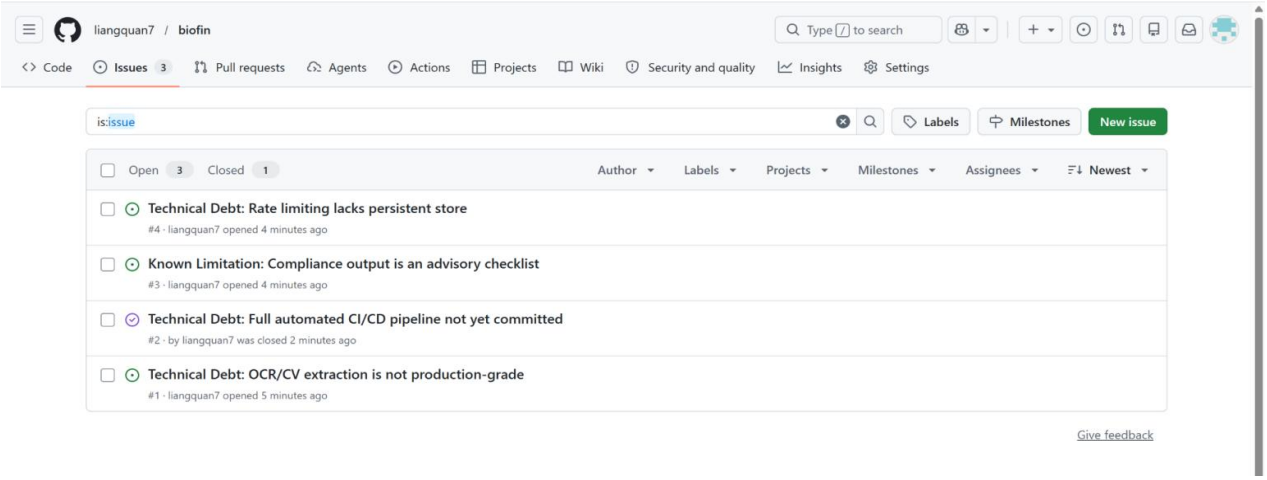
Bug ID	Bug Description	Steps to Reproduce	Console / Error Log	Root Cause Analysis	Fix Commit Hash / PR Link
DEF-01	Dashboard hydration mismatch in BioFin Oracle header caused inconsistent frontend rendering during dashboard load.	1. Open BioFin Oracle dashboard. 2. Load the main page after deployment/build. 3. Observe browser console and header rendering.	Hydration mismatch warning observed in browser console during header rendering.	Header content used client-side dynamic values before hydration was stabilised, causing server-rendered HTML and client-rendered HTML to differ.	Fixed by stabilising BioFin Oracle header rendering. Verified by TC-10 and Figure 1-E. https://github.com/liangquan7/biofin/commit/b9adb54822c35966f29718593ae1bb8f70267ec0
DEF-02	No automated CI workflow existed to verify test/build status before final submission.	1. Push code changes to GitHub. 2. Check the repository Actions tab. 3. Confirm that no automated build/test workflow runs.	No GitHub Actions workflow configured for build-and-test verification.	The repository did not contain a CI workflow file under .github/workflows, so automated regression evidence could not be generated from GitHub.	Fixed by adding the BioFin CI Pipeline with install, npm test, and npm run build steps. Verified by Figure 2-B. https://github.com/liangquan7/biofin/commit/cd7b240bc22bb93693cc0b055cb5675151ffeb5c

4. Known Issues & Deferred Technical Debt

It is normal to ship with minor issues. The following items were identified during development and testing but not resolved before final submission due to scope, time, or infrastructure constraints. These are proactively documented in accordance with UMHackathon engineering maturity expectations.

Note: Proactively documenting minor issues demonstrates engineering maturity and will not negatively impact your score. You WILL be penalised if judges find obvious issues that you hid or failed to identify.

Issue ID	Description & Impact	Reason for Deferral	GitHub Issue Link
TD-01	Technical Debt: Production-grade OCR/CV extraction is deferred. Image routing is supported; if the external vision key is unavailable, the system uses a safe placeholder record. Impact: Image-upload analysis may rely on fallback routing rather than full semantic image extraction.	Full semantic OCR/CV extraction and verification are explicitly out of scope for the hackathon prototype. Production-grade computer vision accuracy requires additional third-party API configuration, validation data, and development time beyond the competition window.	https://github.com/liangquan7/biofin/issues/1
TD-02	Known Limitation: Compliance output is a decision-support checklist — not an official LHDN MyInvois validation result.The SME Compliance & ROI tab renders compliance items and readiness indicators derived from AI analysis. Impact: SME users must not treat the output as a legally validated tax or invoicing compliance result.	Production-grade LHDN MyInvois API integration is explicitly out of scope for a hackathon prototype. The compliance feature is intentionally scoped as an advisory tool.	https://github.com/liangquan7/biofin/issues/3
TD-03	Technical Debt: Rate limiting is a production-readiness placeholder.The rate-limiting layer is present in the architecture but not yet connected to a persistent rate-limit store. Impact: Under repeated rapid requests the serverless route could be called more frequently than intended, increasing API costs.	Free-tier serverless environment does not provide a persistent shared store. Implementing Redis or equivalent requires infrastructure outside hackathon scope.	https://github.com/liangquan7/biofin/issues/4



(Figure 2-E. Known issues and deferred technical debt evidence, GitHub Issues page documents unresolved prototype limitations and future engineering work — Section 4 Known Issues & Deferred Technical Debt)

5. Live Demo / UAT Risks

Every prototype has operational limits. The following risks are documented proactively to ensure fair evaluation within the prototype's defined parameters.

Note: Proactively documenting system boundaries protects your demo score and demonstrates proper risk management. You WILL be significantly penalised if you claim "no risks" and your application crashes or fails unexpectedly during the live demo.

- **Risk 1 — File Upload Size Limit:** Do not upload any individual data file larger than 5 MB. Files above this threshold are rejected at the API boundary before any analysis begins. Representative durian-farm fixture files should remain well below 1 MB each for optimal performance. (Evidence: TC-05, Figure 1-C.)

- **Risk 2 — Analysis Response Time:** The end-to-end analysis pipeline takes approximately 8 seconds under representative payload conditions. Please click the Execute Analysis button only once and wait for the SSE progress events to complete. Clicking repeatedly may trigger multiple concurrent requests and exhaust third-party API quotas. (Evidence: TC-03.)

- **Risk 3 — Browser Compatibility:** BioFin Oracle was developed and tested specifically on modern Chromium-based browsers (Google Chrome, Microsoft Edge). Safari or mobile browsers may experience layout degradation or SSE event handling differences. The live demo should be conducted on Google Chrome for the most reliable evaluation experience.

- **Risk 4 — External API Dependency:** The AI analysis pipeline depends on live GLM and Tavily API services. If either service experiences an outage or rate-limit during the demo, the system will automatically degrade to a safe fallback response — a baseline AnalysisResult will still be rendered and all dashboard tabs will remain navigable. The recommendation will indicate that analysis was performed with limited AI support. (Evidence: TC-08.)

- **Risk 5 — OCR/CV Image Uploads: Uploading images to the envGeoData or bioCropData slots is supported through the image-routing path. Production-grade OCR/CV extraction is deferred; if the external vision key is unavailable, the system uses a safe placeholder record. Judges evaluating image upload functionality should assess this as pipeline routing and fallback handling, not full computer vision accuracy. (Evidence: TC-06, TD-01.)**

- **Risk 6 — Compliance Output Advisory Status:** The SME Compliance & ROI tab displays a compliance checklist and readiness indicators generated by AI analysis. This output is an advisory decision-support tool only — it does not constitute a legally validated LHDN MyInvois compliance result. Judges should evaluate it as a structured decision-support feature. (Evidence: TD-03.)

6. QA Recommendation & Sign-Off

This sign-off confirms that all Final Round testing has been completed, results have been recorded, defects have been traced to root causes and resolved, and known risks have been proactively documented. The BioFin Oracle prototype is submitted in a stable, demo-ready state.

Field	Detail
Prepared By	BioFin Oracle QA Lead — Chen Bing Yan
Reviewed By	BioFin Oracle Technical Lead — Tan Liang Chuan, Chen Bing Yan
System Under Test	UMHackathon 2026 — BioFin Oracle Prototype
Repository	https://github.com/liangquan7/biofin.git
GitHub Issues Board	https://github.com/liangquan7/biofin/issues
Final Round Submission Date	2nd May 2026
Overall Regression Result	PASS — 12/12 test cases passed (100%). 3/3 AI output contract tests passed. Zero P0/P1 defects in upload, API, or fallback flow.
AI Output Pass Rate	PASS — 3/3 prompt/response test pairs met acceptance criteria (TC-AI-01, TC-AI-02, TC-AI-03). Adversarial fencing and fallback paths fully verified.
Critical Bugs (P0/P1)	ZERO — No P0/P1 defects identified in upload workflow, API route, or dashboard fallback flow. Two P2 defects discovered and resolved during development (DEF-01, DEF-02).
API Performance	PASS — End-to-end analysis with four representative CSV files completed in approximately 8 seconds; SSE stage events emitted continuously; no timeout or UI freeze observed.
Sign-Off Status	Approved and finalised for final round submission. All test cases executed; results recorded in Sections 1 through 5. CI/CD workflow has been committed and verified before final round upload.

Important Note: *Using Agile principles, testing was conducted iteratively throughout the development period — not limited to a single end-phase test run. All evidence referenced in this document is retained by the QA lead and is available for judge review upon request.*